Laboratório de Concorrência · Aula 10

Emanuelly P Lopes · RA 24174229-2 · Turma b
Gerado em 09/10/2026, 21:36:40 · tempo registrado no cronômetro: 39:44

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma tarefa depende da outra terminar algo pra poder continuar, tipo numa cozinha o garçom tem que esperar o cozinheiro terminar o prato pra levar. Competição é quando duas querem usar a mesma coisa ao mesmo tempo e só dá pra uma usar, tipo duas pessoas querendo usar o mesmo banheiro.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000, mas como a atividade é sobre condição de corrida acho que vai dar menos.

1ª execução (n = 1.000.000)

1467391

2ª execução (n = 1.000.000)

1017822

3ª execução (n = 1.000.000)

1028655

Saída do programa

.



1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram, deu menos que 2 milhões e cada vez deu um número diferente. Isso acontece porque as threads rodam ao mesmo tempo e o escalonador decide a ordem, que muda toda vez, então a quantidade de somas perdidas também muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Imaginando que o contador ta em 10: a thread 1 lê 10, a thread 2 também lê 10 antes da 1 gravar. As duas somam 1 e ficam com 11. A thread 1 grava 11 e depois a thread 2 grava 11 também. Então teve 2 incrementos mas o valor só subiu 1, a soma da thread 1 se perdeu.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com 1000 deu certo (2000). Acho que é porque a conta é tão rápida que uma thread termina antes da outra começar direito. Isso é perigoso porque nos testes pode parecer que ta tudo certo e o erro só aparecer em produção quando tiver muito acesso, e aí fica difícil de achar o problema.

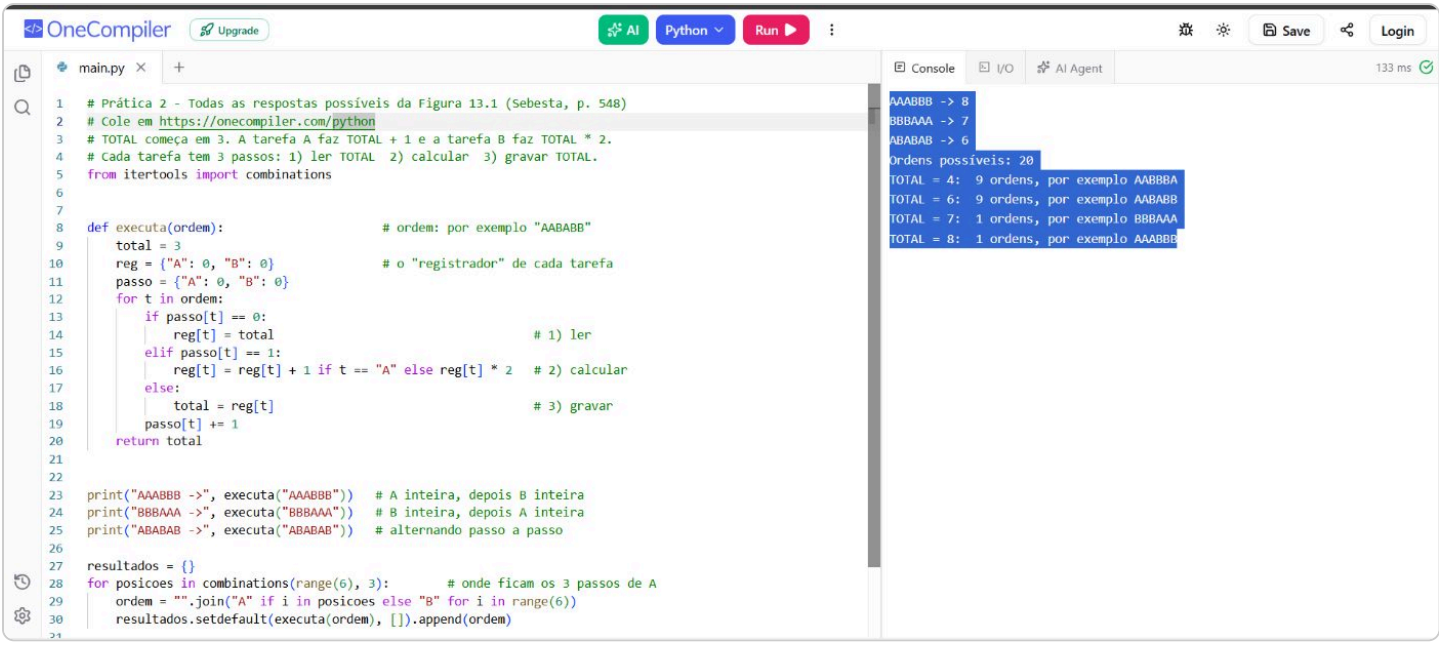
Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BABABA → 4
ABBAAB → 6
BBAABA → 4
AABBAB → 6
BAAABB → 6
BBAAAB → 6
BABBAA → 4
AABBBA → 4
ABABAB → 6
ABBBAA → 4
ABBABA → 4
BBBAAA → 7

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Deu 20 ordens possíveis e 4 resultados diferentes (4, 6, 7 e 8). Só uma ordem deu 8, que é a AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem ABBBAA: A lê o 3, aí B lê 3, multiplica e grava 6. Depois A soma 3+1 = 4 e grava 4 por cima. Como A tinha lido o valor antigo, o que B fez foi perdido.

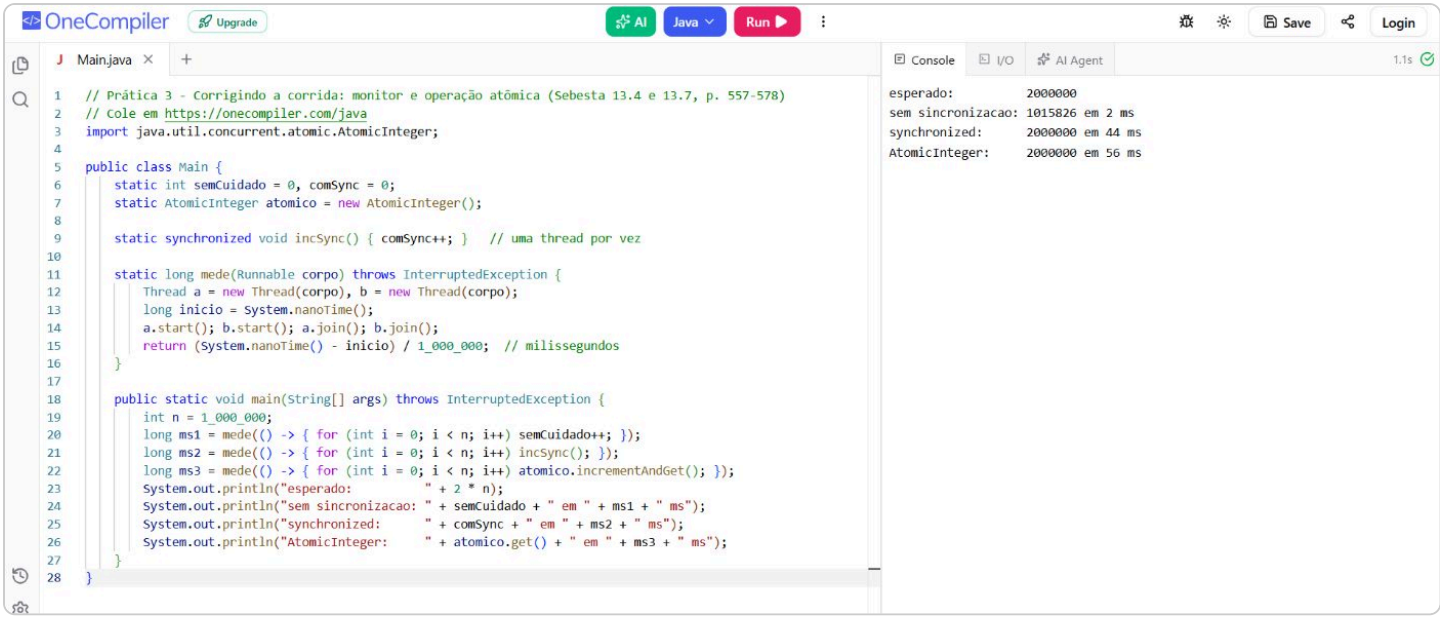
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a sem sincronização vai ser a mais rápida mas vai dar errado. As outras duas devem dar certo mas mais lentas.

Saída do programa

```
Print depois de rodado duas vezes
```



3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

O synchronized e o AtomicInteger deram certo, o sem sincronização deu errado. As certas demoram mais porque as threads tem que ficar esperando uma a outra pra mexer no contador (travar e destravar toda hora), então perde velocidade.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Acho que não, porque a transferência são duas operações (tirar de uma conta e colocar na outra). Mesmo cada saldo sendo atômico, alguma coisa pode acontecer no meio entre as duas. Teria que proteger a transferência inteira de uma vez, tipo travando as duas contas juntas ou usando transação.

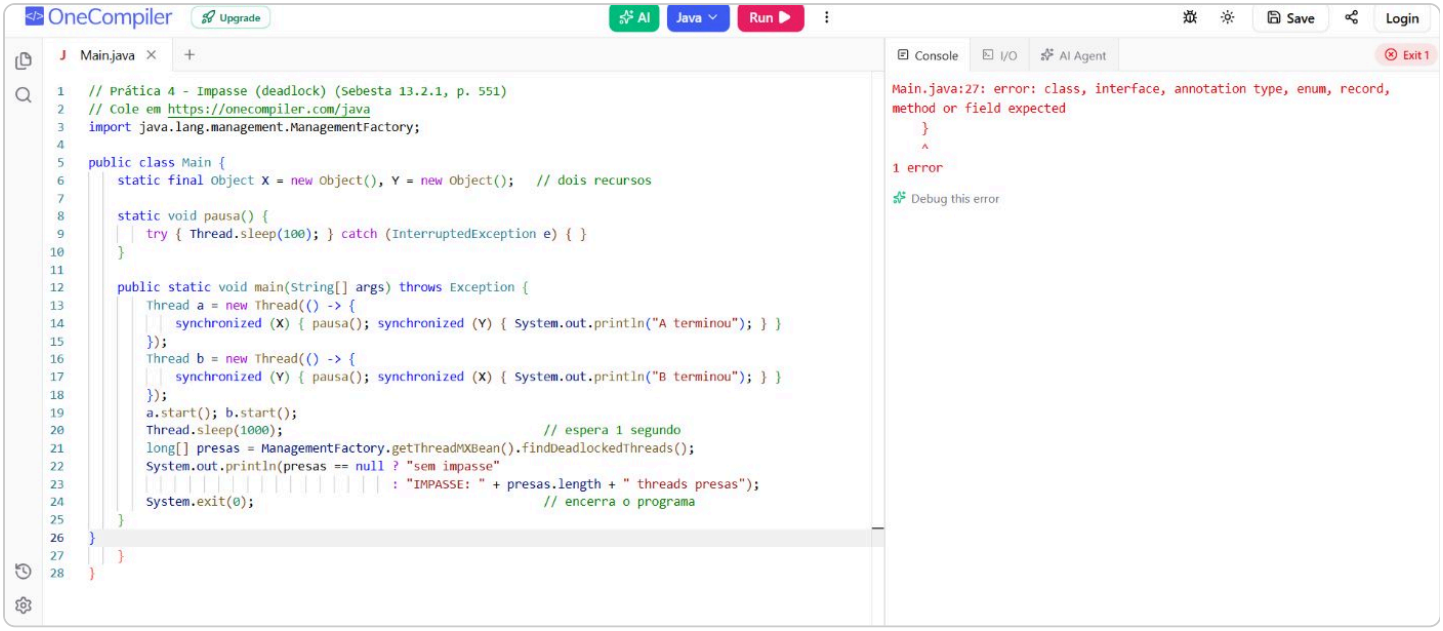
Prática 4: Impasse (deadlock)

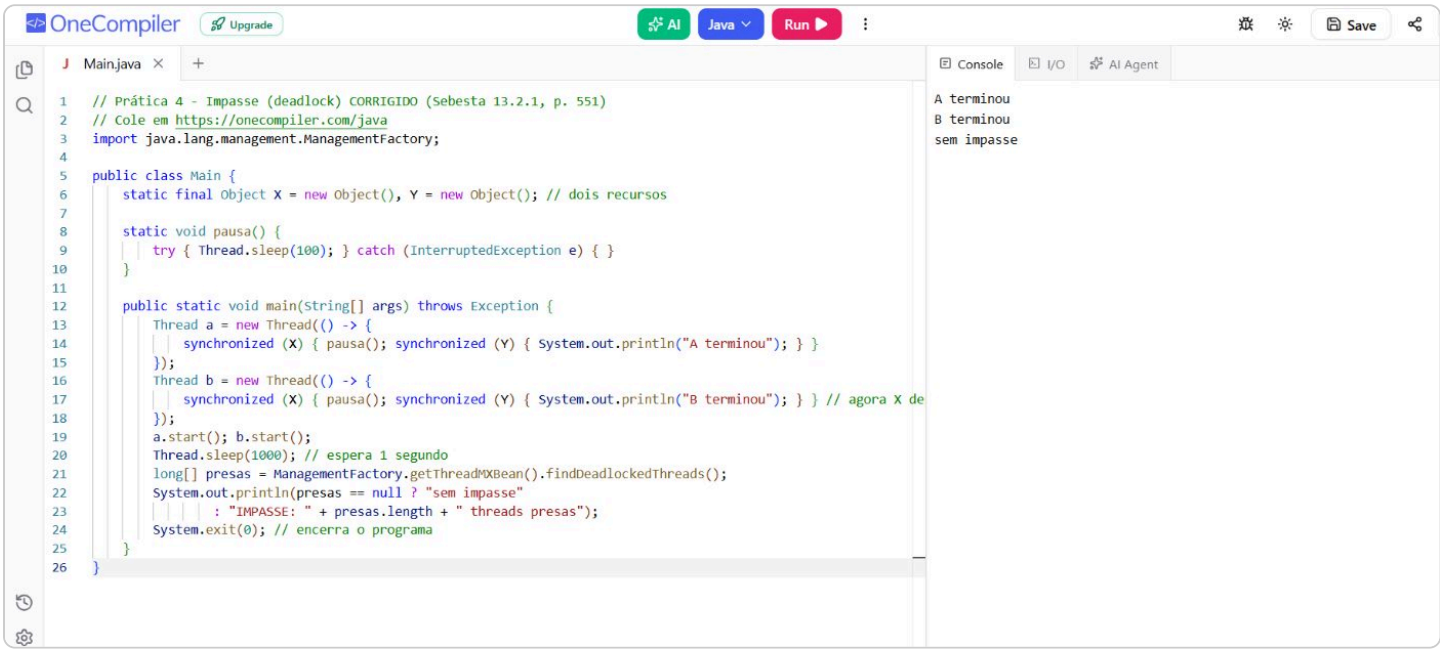
Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que não vão aparecer, porque uma vai ficar esperando a outra.

Saídas (antes e depois da correção)

-





4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pegou o X e ficou esperando o Y, e a B pegou o Y e ficou esperando o X. Como nenhuma solta o que pegou, as duas ficam esperando pra sempre.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque se as duas pegam primeiro o X, a que chegar depois fica esperando sem segurar nada, então a primeira consegue terminar e soltar tudo. Um exemplo real seria num banco com duas transferências ao mesmo tempo, uma de A pra B e outra de B pra A.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 uns 200 ms, com 1 uns 600 ms e com 6 uns 100 ms.

Tempo com 1 trabalhador

levou 600ms

Tempo com 3 trabalhadores

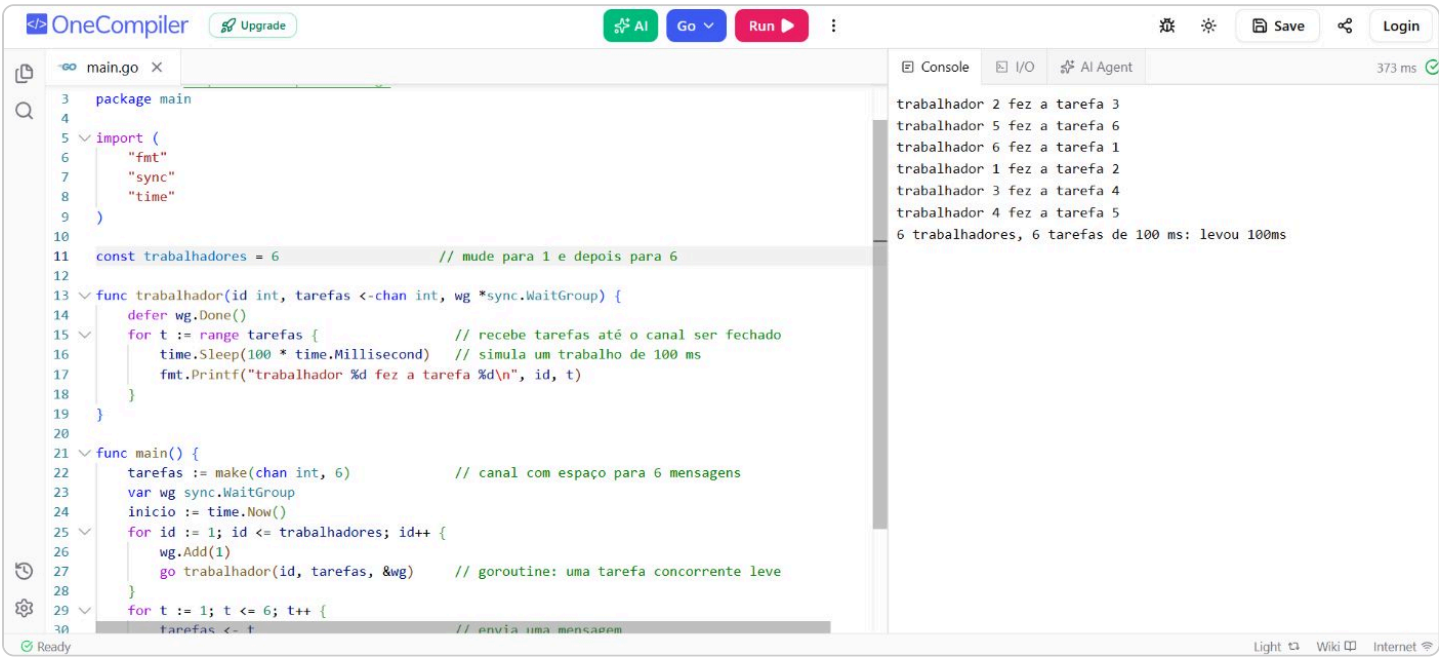
levou 200ms

Tempo com 6 trabalhadores

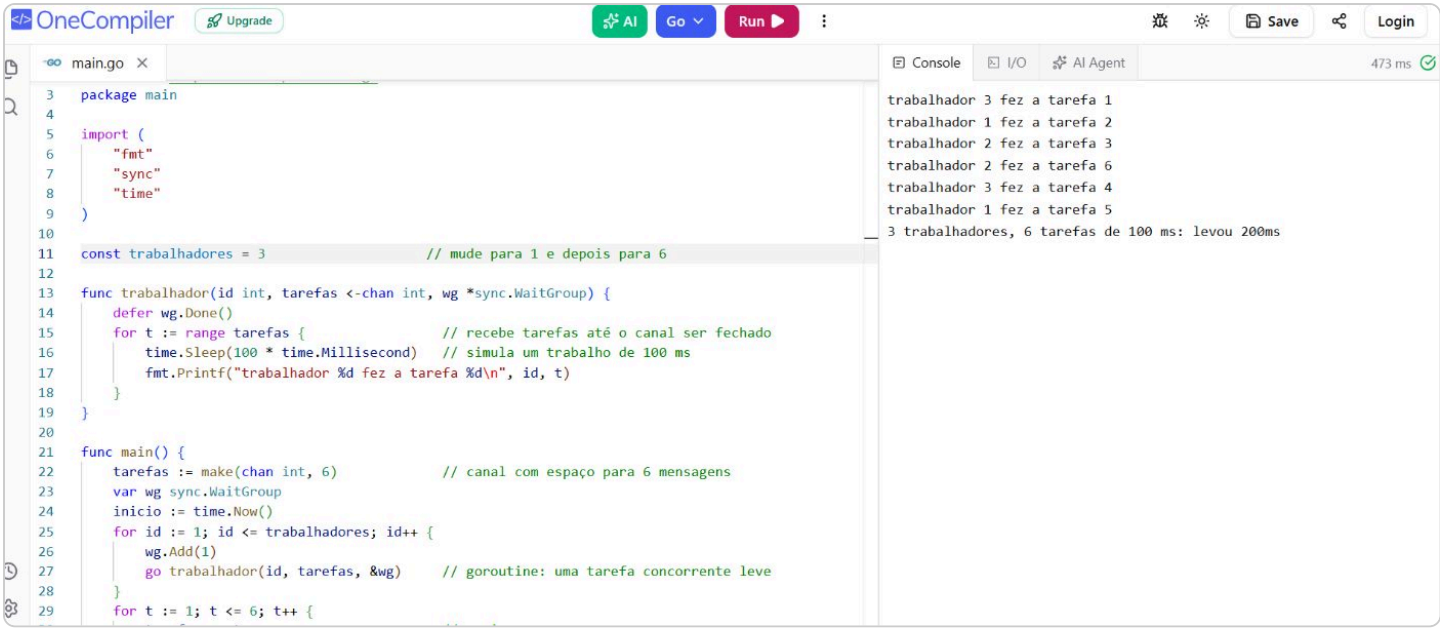
levou 100ms

Saída do programa

.





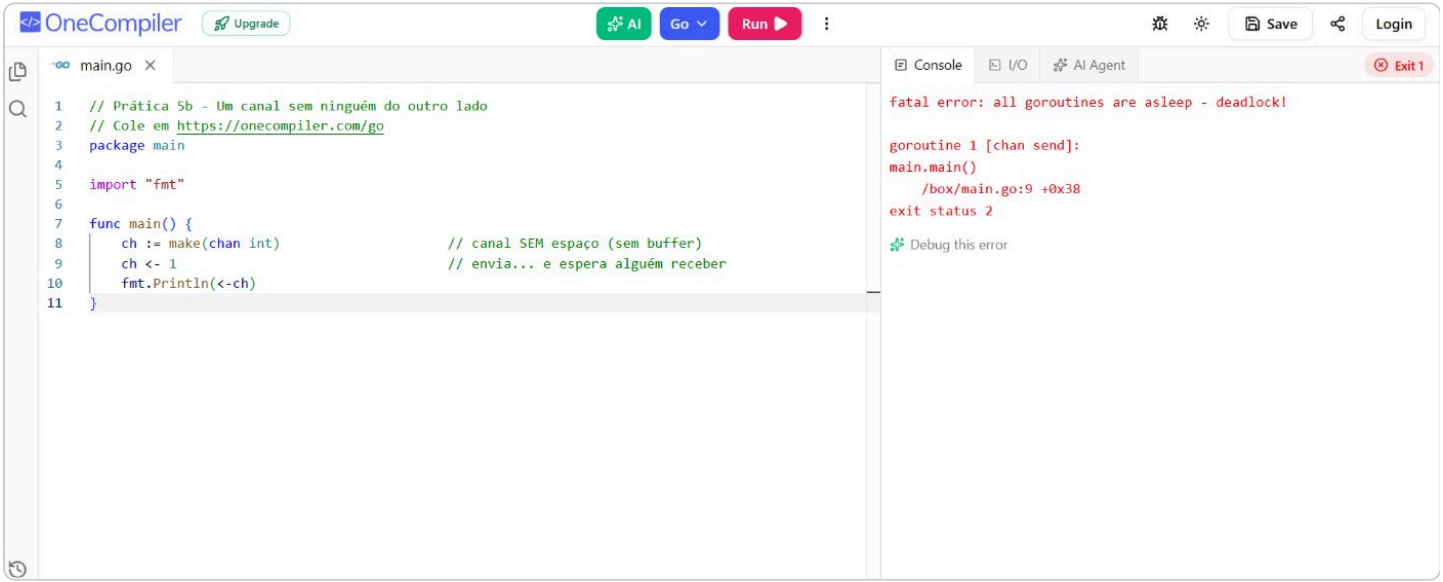


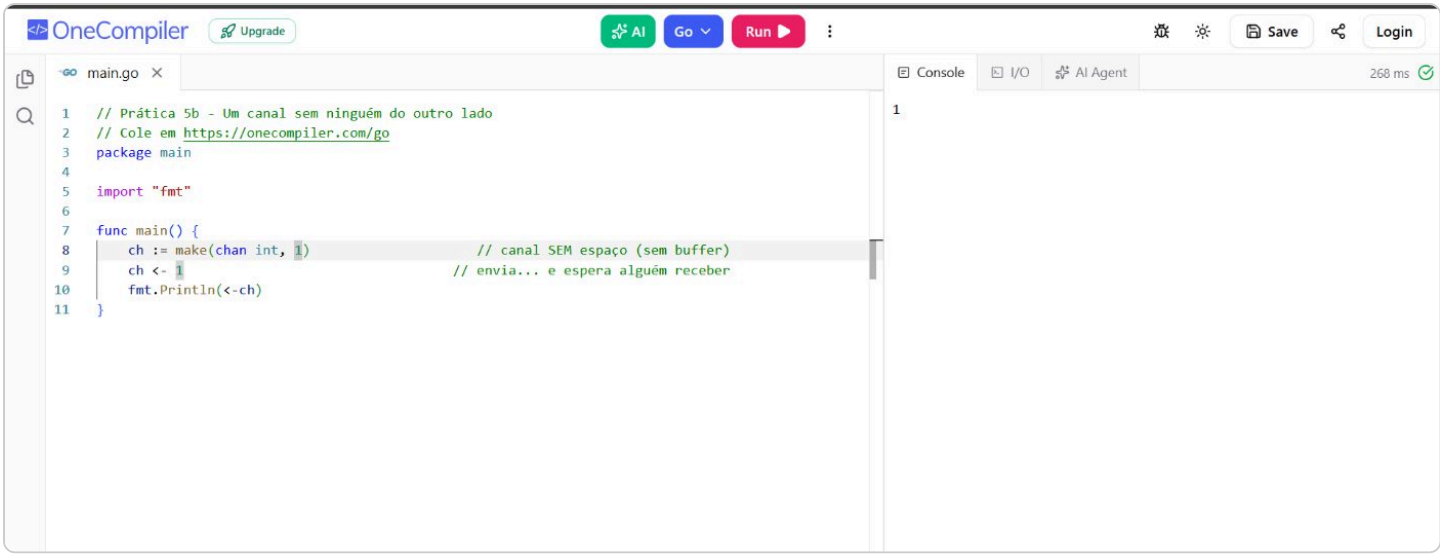
5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador ele faz as 6 tarefas uma depois da outra (600 ms), com 3 cada um faz 2 (200 ms) e com 6 todo mundo faz uma ao mesmo tempo (100 ms). Não precisa de trava porque eles não mexem na mesma variável, as tarefas vão pelo canal e cada uma só vai pra um trabalhador.

Saídas (antes e depois da troca)

-





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Trava porque o canal não tem espaço, então o envio fica esperando alguém receber, mas não tem mais ninguém pra receber, só a própria main que ta parada ali. Aí dá o erro de deadlock. Com espaço 1 o valor fica guardado no canal e o programa continua, depois ele mesmo lê o valor.

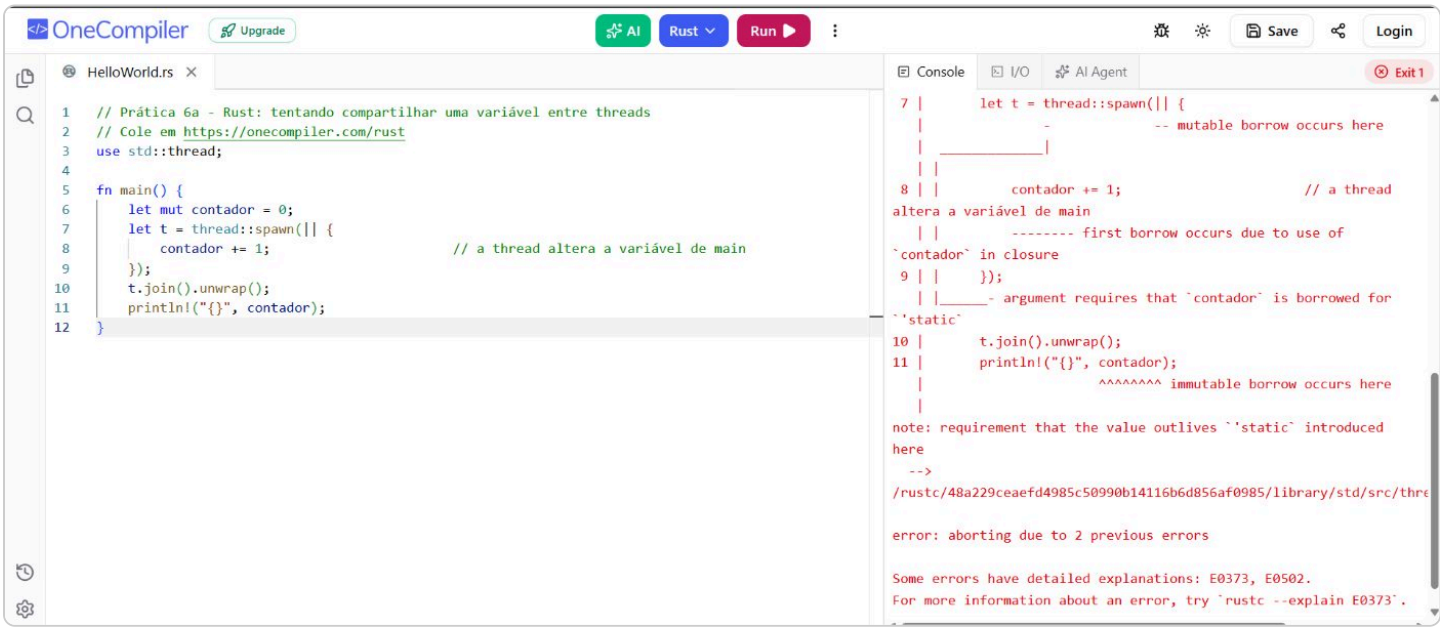
Prática 6: O compilador como guarda

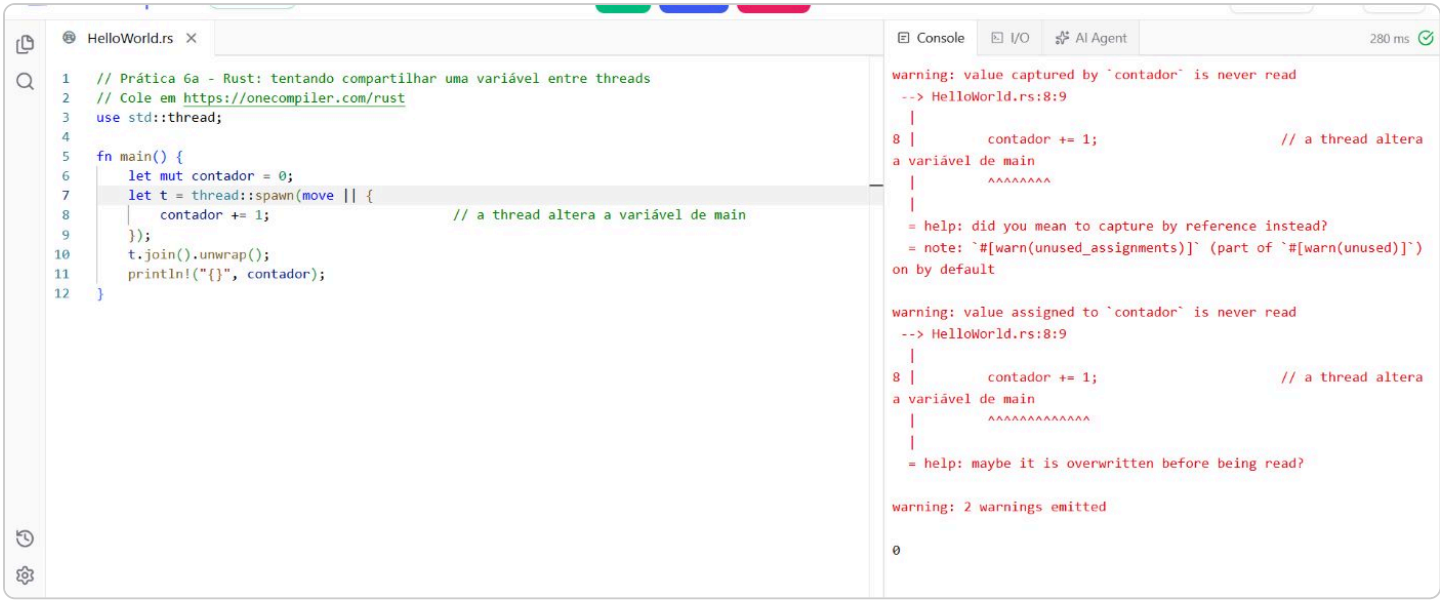
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que vai dar erro de compilação.

Saídas (o erro e o resultado com move)

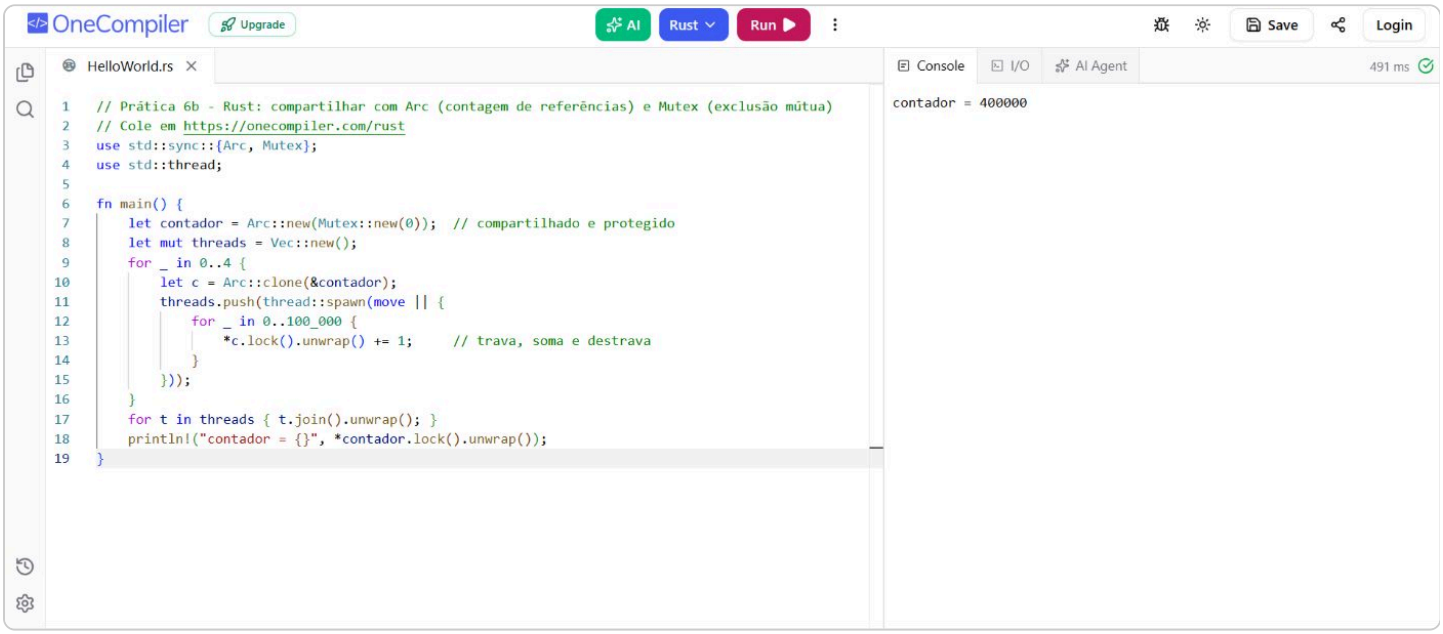
.





Saída do programa

.



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Porque com o move a thread leva uma cópia do número e soma na cópia, então o contador original da main continua 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

No Java o erro só apareceu rodando o programa (e com n pequeno nem apareceu). No Rust apareceu já na hora de compilar. A vantagem é que o erro é pego antes, o custo é que é mais difícil de programar e precisa usar coisas como Arc e Mutex.

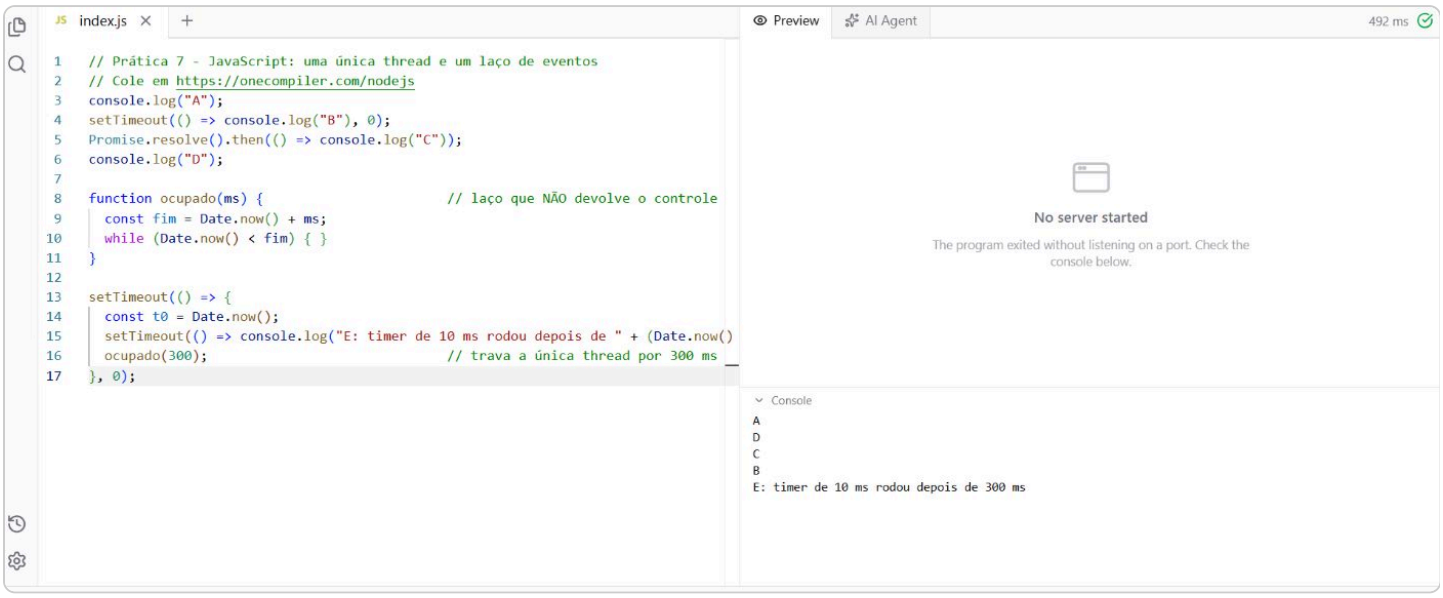
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, B, C, D

Saída do programa

.



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. Eu errei a previsão. O B sai depois porque o setTimeout vai pra uma fila e só roda quando o código normal acaba, e as promessas (C) tem prioridade antes dos timers.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque a função ocupado ficou travando a thread por 300 ms, e como o JS só tem uma thread o timer teve que esperar. Num servidor Node isso quer dizer que se tiver um cálculo pesado todos os outros usuários ficam esperando.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Condição de corrida, as duas pessoas podem ver que tem 1 lugar e as duas conseguirem comprar o mesmo ingresso. Pra evitar tem que fazer a verificação e a reserva juntas, de forma atômica, tipo travando o registro no banco de dados enquanto um compra.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Me surpreendeu que um contador++ simples pode dar resultado errado.